Generator Comprehensions for Memory Optimization

(expression for item in iterable) yields values one at a time – constant memory however big the data.

List vs generator expression

```
import sys
squares_list = [n * n for n in range(1_000_000)]
squares_gen = (n * n for n in range(1_000_000))
# ~8 MB
print(sys.getsizeof(squares_list) > 1_000_000)
# ~200 bytes
print(sys.getsizeof(squares_gen) < 1_000)
```

OUTPUT

```
True
True
```

	List `[...]`	Generator `(...)`
Memory	stores every item	stores only its state
First item	after building all	immediately
Reuse	any number of times	**one pass** only
`len()` / index	yes	no
Best for	reuse, random access	streaming, one-time use

Feed aggregate functions directly

```
nums = range(1, 6)
# no extra () needed
print(sum(n * n for n in nums))
print(max(len(w) for w in ["a", "bbb", "cc"]))
print(any(n > 4 for n in nums))
print(", ".join(str(n) for n in nums))
```

OUTPUT

```
55
3
True
1, 2, 3, 4, 5
```

Same thing as a generator function

```
def squares(n):
    for i in range(n):
        yield i * i

print(list(squares(4)))
print(list(i * i for i in range(4)))
```

OUTPUT

```
[0, 1, 4, 9]
[0, 1, 4, 9]
```

Chain them: a lazy pipeline

```
nums = range(10)
evens = (n for n in nums if n % 2 == 0)
# nothing computed yet
squares = (n * n for n in evens)
# work happens here
print(sum(squares))
```

OUTPUT

```
120
```

One-shot behaviour

```
gen = (n * 2 for n in range(3))
print(next(gen), next(gen))
print(list(gen))              # only what is left
print(list(gen))              # exhausted -> empty
```

OUTPUT

```
0 2
[4]
[]
```

Short-circuiting saves work

```
def check(n):
    print("checked", n)
    return n > 1

print(any(check(n) for n in [1, 2, 3, 4]))
```

OUTPUT

```
checked 1
checked 2
True
```

Stream a file, keep memory flat

```
import io
log = io.StringIO("ok\nERROR a\nok\nERROR b\n")
# same idea:  with open("big.log") as log: ...
errors = (ln.strip() for ln in log if "ERROR" in ln)
print(list(errors))
```

OUTPUT

```
['ERROR a', 'ERROR b']
```

PITFALL A generator can't be indexed, `len()`-ed or replayed. Convert to a list only when the result is small, and don't change the source while iterating.

REMEMBER Consumed once and possibly huge? → generator expression. Need to reuse or index? → list comprehension.